



FACULTY OF ENGINEERING

**A REPORT ABOUT USING THE ALGORITHM DESIGN PROCEDURE OF
DEVELOPING FLOW CHARTS, PSEUDOCODE AND MATLAB CODE,
DEVELOPING A CLASS AND A GRAPHIC USER INTERFACE FOR ALL THE
NUMERICAL METHODS.**

COURSE UNIT: COMPUTER PROGRAMMING

LECTURER: Mr. MASERUKA BENEDICTO

GROUP H

SUBMITTED BY

1. Nakimolo Benard
2. Tumubwine Promise
3. Lakwo George
4. Malinga Robert
5. Achobi Esther
6. Okabo Samson
7. Nabwire Aisha Wadindi

DATE OF SUBMISSION:

SUBMITTED TO:

Contents

ACKNOWLEDGEMENT	4
ABSTRACT.....	5
DEDICATION.....	6
DECLARATION	7
APPROVAL	8
LIST OF ACRONYMS/ ABBREVIATIONS	9
CHAPTER ONE: INTRODUCTION.....	10
1.1 Background	10
1.2 Historical Development	10
CHAPTER 2: STUDY METHODOLOGY	11
2:1 Introduction.....	11
2.2 Discussion chapter	11
2.3 Steps involved.....	11
Step 1: Open MATLAB.....	11
Step 2: Create a New Script.....	11
Step 3: Create the Class Code.....	11
Step 4: Create the GUI Code	11
Step 5: Run the GUI.....	11
STUDY COVERAGE	13
Introduction.....	13
CHAPTER 3: NEWTON-RAPHSON METHOD.....	13
3.1 Classes code	13
3.2 GUI code	14
3.3 Figure and plot	16
CHAPTER 4: SECANT METHOD	18
4.1 Class code	18
4.2 GUI code	19
4.3 Figure and plots.....	21
CHAPTER 5: BISECTION METHOD	22
5.1 Class code	22
5.2 GUI code	23
5.3 Figure and plots.....	25
CHAPTER 6: FIXED-POINT ITERATION.....	26
6.1 Class code	26
6.2 GUI code.....	26
6.3 figure and plot	28
CHAPTER 7: TRAPEZOIDAL METHOD	29

7.1 Class code	29
7.2 GUI CODE.....	30
7.3 figure and plot	31
CHAPTER 8: CONCLUSION AND RECOMMENDATION	32
8.1 Recommendations for GUI Design.....	32
8.2 Conclusions on GUI Design	32
References.....	33

ACKNOWLEDGEMENT

First and foremost, we would like to thank the Almighty God for giving us the strength to carry on with our research in group H. We would love to extend our gratitude to all the persons with whose help we managed to make it this far

The willingness of each one of us to invest time and provide constructive feedback has been immensely valuable in this assignment. Finally, we would like to express our gratitude to all the sources and references that have been cited in this report

ABSTRACT

We started our research from 25th, October, 2024 in the university library out of which we were exposed to various numerical methods in the different sections through Group H, where we obtained research on the algorithm design procedure of developing flow charts, pseudocode and MATLAB code, developing a class and a graphic user interface for all the numerical methods. Information we used was obtained from YouTube and other MATLAB textbooks. Plotting the time, accuracy graphs and the number of steps followed graph. We achieved this through division of labor, different members were assigned different numerical methods in order to ease the work and to save time.

DEDICATION.

We dedicate this report to all the individuals especially Group H members, who have been there for us in the process of formulating and producing this report. To our lecturer Mr. Maseruka Benedicto whose guidance and expertise have been invaluable, your mentorship and insightful feedback have shaped our understanding.

DECLARATION

We hereby certify and confirm that the information in this report is out of our own efforts, research and it has never been submitted in any institution for any academic award

NAME	SIGNATURE
Nakimolo Benard	
Tumubwine Promise	
Lakwo George	
Malinga Robert	
Achobi Esther	
Okabo Samson	
Nabwire Aisha Wadindi	

APPROVAL

We are presenting this report which has been written and produced under our efforts. We carried out research using the algorithm design procedure of developing flowchart, pseudocode and the MATLAB code, Developing a class and a graphical user interface for the five different numerical methods.

LIST OF ACRONYMS/ ABBREVIATIONS.

1. GUI - Graphical user interface.
2. MATLAB - Matrix Laboratory.

CHAPTER ONE: INTRODUCTION

1.1 Background.

MATLAB, which stands for matrix laboratory, is a high-performance programming language and environment designed primarily for technical computing. Its origins trace back to the late 1970s when Cleve Moler, a professor of computer science, developed it to provide his students with easy access to mathematical software libraries without requiring them to learn Fortran.

1.2 Historical Development

- **Initial Development:** The first version of MATLAB was created in Fortran in the late 1970s as a simple interactive matrix calculator. This early iteration included basic matrix operations and was built on top of two significant mathematical libraries: LINPACK and EISPACK, which were developed for numerical linear algebra and eigenvalue problems, respectively.
- **Commercial Launch:** MATLAB was officially launched as a commercial product in 1984 by MathWorks, a company founded by Moler along with Jack Little and Steve Bangert. This marked the transition from a simple calculator to a comprehensive programming environment. The software was reimplemented in C, enhancing its capabilities with the addition of user-defined functions, toolboxes, and graphical interfaces.
- **Expansion and Toolboxes:** Over the years, MATLAB has expanded significantly. By the late 1980s, it had introduced several specialized toolboxes for various applications, including control systems and signal processing. The introduction of the *Simulink* environment further allowed users to model and simulate dynamic systems graphically.
- **Modern Enhancements:** Recent versions of MATLAB have introduced features like the *Live Editor*, which allows users to create interactive documents that combine code, output, and formatted text. This evolution reflects MATLAB's ongoing adaptation to meet the needs of its diverse user base across academia and industry.

CHAPTER 2: STUDY METHODOLOGY

2:1 Introduction

At the start time of the research, we divided ourselves accordingly depending on the numerical methods including the callback functions, classes and the GUI of every numerical method talked about. At least each member had to look for information about a given method and how its GUI interface works.

Names and method researched

Names	Numerical method
Nakimolo Benard	Trapezoidal and developing the GUI
Tumubwine Promise	Fixed point iteration
Achobi Esther and Malinga Robert	secant
Okabo Samson	Newton Raphson method
Lakwo George and Nabwire Aisha Wadindi	Bisection method

2.2 Discussion chapter

We were able to run the MATLAB code and use the GUI to find roots using the different numerical methods.

2.3 Steps involved.

Step 1: Open MATLAB

Launch MATLAB on your computer.

Step 2: Create a New Script

In the MATLAB environment, go to the Home tab.

Click on New Script to open the editor.

Step 3: Create the Class Code

Write the numerical method class code.

Save the script with the name numerical method.m. Make sure to save it in a directory that is in your MATLAB path or the current working directory.

Step 4: Create the GUI Code

Open another new script in the MATLAB editor

Save this script with the name numerical method name GUI.m.

Step 5: Run the GUI

In the MATLAB Command Window, type the following command to run the GUI

We discussed the various numerical methods involving MATLAB code and use of the GUI including;

- Newton's Raphson method
- Fixed point iteration
- Secant method
- Trapezoidal method
- Bisection method

STUDY COVERAGE

Introduction

We used one equation for all the numerical methods in order to obtain accuracy graphs and comparison.

The formula is $f(x) = 2^x - 5x + 2 = 0$

CHAPTER 3: NEWTON-RAPHSON METHOD

3.1 Classes code

```
classdef NewtonRaphson
    properties
        functionHandle
        derivativeHandle
        initialGuess
        tolerance
        maxIterations
    end
    methods
        function obj = NewtonRaphson(f, df, x0, tol, max_iter)
            obj.functionHandle = f;
            obj.derivativeHandle = df;
            obj.initialGuess = x0;
            obj.tolerance = tol;
            obj.maxIterations = max_iter;
        end

        function [root, steps, times] = findRoot(obj)
            n = 0;
            x_n = obj.initialGuess;
            steps = [];
            times = [];

            while n < obj.maxIterations
                start_time = tic; % Start timing
                f_x_n = obj.functionHandle(x_n);
                df_x_n = obj.derivativeHandle(x_n);

                if abs(f_x_n) < obj.tolerance
                    root = x_n;
                    return;
                end

                x_n = x_n - f_x_n / df_x_n;
                steps(end + 1) = x_n; % Store the current approximation
                times(end + 1) = toc(start_time); % Record time taken
            end
        end
    end
end
```

```

        n = n + 1;
    end

    root = 'Max iterations reached';
end
end
end

```

3.2 GUI code

```

function NewtonRaphsonGUI
    % Create figure
    f = figure('Position', [100, 100, 500, 500], 'MenuBar', 'none', ...
        'Name', 'Newton-Raphson Method', 'NumberTitle', 'off');

    % Create UI components
    uicontrol('Style', 'text', 'Position', [20, 420, 120, 20], 'String', 'Function
f(x):');
    functionInput = uicontrol('Style', 'edit', 'Position', [150, 420, 320, 20]);

    uicontrol('Style', 'text', 'Position', [20, 380, 120, 20], 'String',
'Derivative f'(x):');
    derivativeInput = uicontrol('Style', 'edit', 'Position', [150, 380, 320, 20]);

    uicontrol('Style', 'text', 'Position', [20, 340, 120, 20], 'String', 'Initial
Guess:');
    initialGuessInput = uicontrol('Style', 'edit', 'Position', [150, 340, 320,
20]);

    uicontrol('Style', 'text', 'Position', [20, 300, 120, 20], 'String',
'Tolerance:');
    toleranceInput = uicontrol('Style', 'edit', 'Position', [150, 300, 320, 20]);

    uicontrol('Style', 'text', 'Position', [20, 260, 120, 20], 'String', 'Max
Iterations:');
    maxIterInput = uicontrol('Style', 'edit', 'Position', [150, 260, 320, 20]);

    resultDisplay = uicontrol('Style', 'text', 'Position', [20, 220, 400, 20],
'String', '');

    uicontrol('Style', 'pushbutton', 'String', 'Find Root', ...
        'Position', [200, 180, 100, 30], ...
        'Callback', @(src, event) calculateRoot());

    % Callback function
    function calculateRoot()
        % Get inputs
        f_str = get(functionInput, 'String');
        df_str = get(derivativeInput, 'String');
        x0 = str2double(get(initialGuessInput, 'String'));
        tol = str2double(get(toleranceInput, 'String'));
    end
end

```

```

max_iter = str2double(get(maxIterInput, 'String'));

% Convert string function to function handle
f = str2func(['@(x) ' f_str]);
df = str2func(['@(x) ' df_str]);

% Create Newton-Raphson object
nr = NewtonRaphson(f, df, x0, tol, max_iter);
[root, steps, times] = nr.findRoot();

% Display result
set(resultDisplay, 'String', ['Root: ' num2str(root)]);

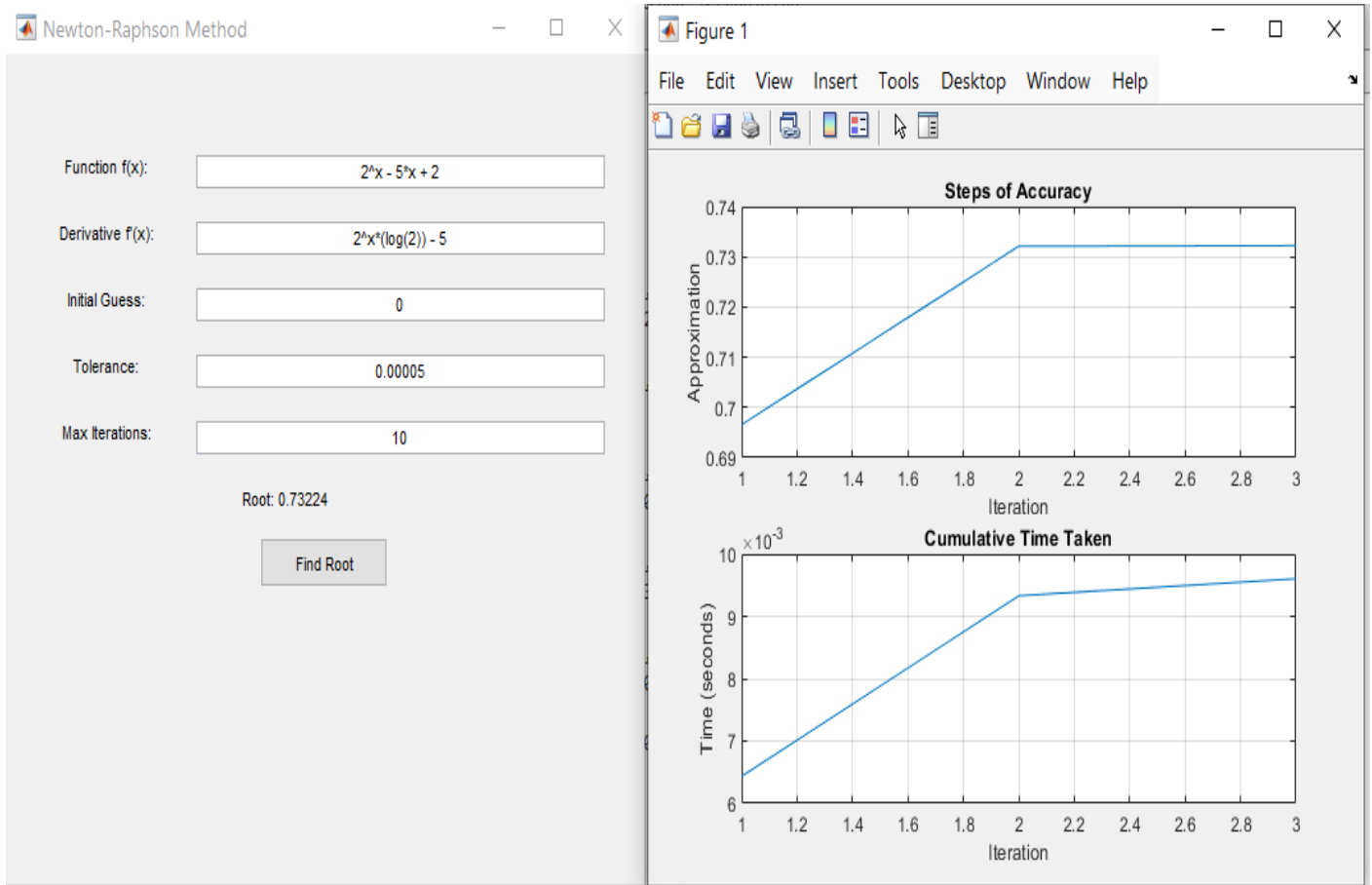
% Plotting steps of accuracy
figure;
subplot(2, 1, 1);
plot(steps);
title('Steps of Accuracy');
xlabel('Iteration');
ylabel('Approximation');
grid on;

% Plotting time taken
subplot(2, 1, 2);
plot(cumsum(times));
title('Cumulative Time Taken');
xlabel('Iteration');
ylabel('Time (seconds)');
grid on;
end
end

```

3.3 Figure and plot

The figure below shows where the user inputs the function, derivative of the function, initial guess, tolerance and the maximum number of iterations. After when a user clicks the find root button the root is displayed.



CHAPTER 4: SECANT METHOD

4.1 Class code

```
classdef SecantMethod
    properties
        functionHandle
        initialGuess1
        initialGuess2
        tolerance
        maxIterations
    end

    methods
        function obj = SecantMethod(f, x0, x1, tol, max_iter)
            obj.functionHandle = f;
            obj.initialGuess1 = x0;
            obj.initialGuess2 = x1;
            obj.tolerance = tol;
            obj.maxIterations = max_iter;
        end

        function [root, steps, times] = findRoot(obj)
            n = 0;
            x0 = obj.initialGuess1;
            x1 = obj.initialGuess2;
            steps = [];
            times = [];

            while n < obj.maxIterations
                start_time = tic; % Start timing
                f_x0 = obj.functionHandle(x0);
                f_x1 = obj.functionHandle(x1);

                if abs(f_x1) < obj.tolerance
                    root = x1;
                    return;
                end

                x2 = x1 - (f_x1 * (x1 - x0)) / (f_x1 - f_x0);
                steps(end + 1) = x2; % Store the current approximation
                times(end + 1) = toc(start_time); % Record time taken

                % Update variables for next iteration
                x0 = x1;
                x1 = x2;
            end
        end
    end
end
```

```

        n = n + 1;
    end

    root = 'Max iterations reached';
end
end
end
end
4.2 GUI code
function SecantMethodGUI
% Create figure
f = figure('Position', [100, 100, 500, 500], 'MenuBar', 'none', ...
    'Name', 'Secant Method', 'NumberTitle', 'off');

% Create UI components
uicontrol('Style', 'text', 'Position', [20, 420, 120, 20], 'String', 'Function
f(x):');
functionInput = uicontrol('Style', 'edit', 'Position', [150, 420, 320, 20]);

uicontrol('Style', 'text', 'Position', [20, 380, 120, 20], 'String', 'Initial
Guess x0:');
initialGuess1Input = uicontrol('Style', 'edit', 'Position', [150, 380, 320,
20]);

uicontrol('Style', 'text', 'Position', [20, 340, 120, 20], 'String', 'Initial
Guess x1:');
initialGuess2Input = uicontrol('Style', 'edit', 'Position', [150, 340, 320,
20]);

uicontrol('Style', 'text', 'Position', [20, 300, 120, 20], 'String',
'Tolerance:');
toleranceInput = uicontrol('Style', 'edit', 'Position', [150, 300, 320, 20]);

uicontrol('Style', 'text', 'Position', [20, 260, 120, 20], 'String', 'Max
Iterations:');
maxIterInput = uicontrol('Style', 'edit', 'Position', [150, 260, 320, 20]);

resultDisplay = uicontrol('Style', 'text', 'Position', [20, 220, 400, 20],
'String', '');

uicontrol('Style', 'pushbutton', 'String', 'Find Root', ...
    'Position', [200, 180, 100, 30], ...
    'Callback', @(src, event) calculateRoot());

% Callback function
function calculateRoot()
% Get inputs
f_str = get(functionInput, 'String');
x0 = str2double(get(initialGuess1Input, 'String'));
x1 = str2double(get(initialGuess2Input, 'String'));
tol = str2double(get(toleranceInput, 'String'));

```

```

max_iter = str2double(get(maxIterInput, 'String'));

% Convert string function to function handle
f = str2func(['@(x) ' f_str]);

% Create SecantMethod object
secant = SecantMethod(f, x0, x1, tol, max_iter);
[root, steps, times] = secant.findRoot();

% Display result
set(resultDisplay, 'String', ['Root: ' num2str(root)]);

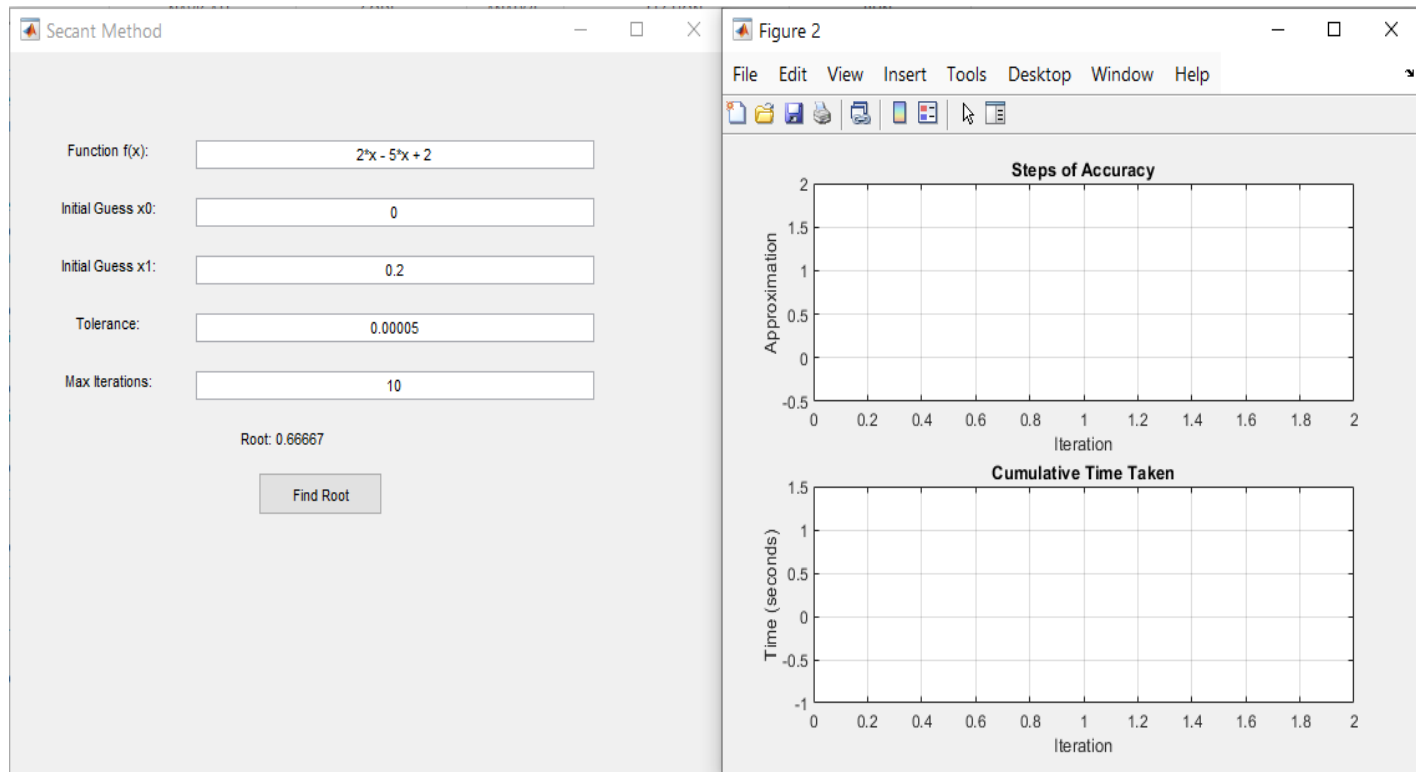
% Plotting steps of accuracy
figure;
subplot(2, 1, 1);
plot(steps);
title('Steps of Accuracy');
xlabel('Iteration');
ylabel('Approximation');
grid on;

% Plotting time taken
subplot(2, 1, 2);
plot(cumsum(times));
title('Cumulative Time Taken');
xlabel('Iteration');
ylabel('Time (seconds)');
grid on;
end
end

```

4.3 Figure and plots

The figure below shows where the user inputs the function, initial guess x_0 , initial guess x_1 , tolerance and the maximum number of iterations. After when a user clicks the find root button the root is displayed.



CHAPTER 5: BISECTION METHOD

5.1 Class code

```
classdef BisectionMethod
    properties
        functionHandle
        a
        b
        tolerance
        maxIterations
    end

    methods
        function obj = BisectionMethod(f, a, b, tol, max_iter)
            obj.functionHandle = f;
            obj.a = a;
            obj.b = b;
            obj.tolerance = tol;
            obj.maxIterations = max_iter;
        end

        function [root, steps, times] = findRoot(obj)
            % Check if there is a root in the interval
            if obj.functionHandle(obj.a) * obj.functionHandle(obj.b) > 0
                root = 'No root in the interval';
                steps = [];
                times = [];
                return;
            end

            n = 0;
            steps = [];
            times = [];

            while n < obj.maxIterations
                start_time = tic; % Start timing
                c = (obj.a + obj.b) / 2;
                f_c = obj.functionHandle(c);

                if abs(f_c) < obj.tolerance
                    root = c;
                    return;
                end

                steps(end + 1) = c; % Store the current approximation
                times(end + 1) = toc(start_time); % Record time taken

                if obj.functionHandle(obj.a) * f_c < 0
```

```

        obj.b = c; % Root is in the left subinterval
    else
        obj.a = c; % Root is in the right subinterval
    end

    n = n + 1;
end

root = 'Max iterations reached';
end
end
end

```

5.2 GUI code

```

function BisectionMethodGUI
% Create figure
f = figure('Position', [100, 100, 500, 500], 'MenuBar', 'none', ...
    'Name', 'Bisection Method', 'NumberTitle', 'off');

% Create UI components
uicontrol('Style', 'text', 'Position', [20, 420, 120, 20], 'String', 'Function
f(x):');
functionInput = uicontrol('Style', 'edit', 'Position', [150, 420, 320, 20]);

uicontrol('Style', 'text', 'Position', [20, 380, 120, 20], 'String', 'Lower
Bound (a):');
lowerBoundInput = uicontrol('Style', 'edit', 'Position', [150, 380, 320, 20]);

uicontrol('Style', 'text', 'Position', [20, 340, 120, 20], 'String', 'Upper
Bound (b):');
upperBoundInput = uicontrol('Style', 'edit', 'Position', [150, 340, 320, 20]);

uicontrol('Style', 'text', 'Position', [20, 300, 120, 20], 'String',
'Tolerance:');
toleranceInput = uicontrol('Style', 'edit', 'Position', [150, 300, 320, 20]);

uicontrol('Style', 'text', 'Position', [20, 260, 120, 20], 'String', 'Max
Iterations:');
maxIterInput = uicontrol('Style', 'edit', 'Position', [150, 260, 320, 20]);

resultDisplay = uicontrol('Style', 'text', 'Position', [20, 220, 400, 20],
'String', '');

uicontrol('Style', 'pushbutton', 'String', 'Find Root', ...
    'Position', [200, 180, 100, 30], ...
    'Callback', @(src, event) calculateRoot());

% Callback function
function calculateRoot()
    % Get inputs
    f_str = get(functionInput, 'String');

```

```

a = str2double(get(lowerBoundInput, 'String'));
b = str2double(get(upperBoundInput, 'String'));
tol = str2double(get(toleranceInput, 'String'));
max_iter = str2double(get(maxIterInput, 'String'));

% Convert string function to function handle
f = str2func(['@(x) ' f_str]);

% Create BisectionMethod object
bisection = BisectionMethod(f, a, b, tol, max_iter);
[root, steps, times] = bisection.findRoot();

% Display result
set(resultDisplay, 'String', ['Root: ' num2str(root)]);

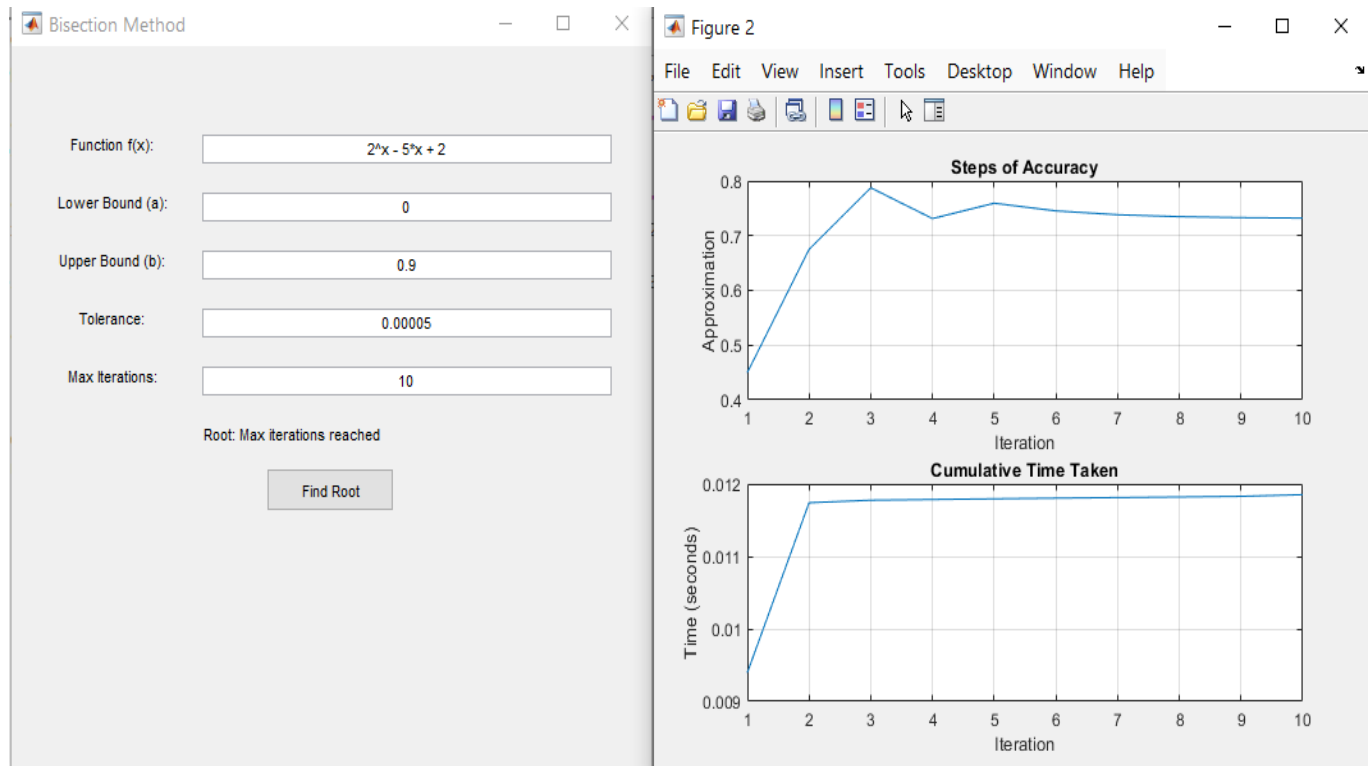
% Plotting steps of accuracy
figure;
subplot(2, 1, 1);
plot(steps, 'o');
title('Steps of Accuracy');
xlabel('Iteration');
ylabel('Approximation');
grid on;

% Plotting time taken
subplot(2, 1, 2);
plot(cumsum(times), 'o');
title('Cumulative Time Taken');
xlabel('Iteration');
ylabel('Time (seconds)');
grid on;
end
end

```


5.3 Figure and plots

The figure below shows where the user inputs the function, initial guess x_0 , initial guess x_1 , tolerance and the maximum number of iterations. After when a user clicks the find root button the root is displayed.



CHAPTER 6: FIXED-POINT ITERATION

6.1 Class code

```
classdef FixedPointMethod
    properties
        FunctionHandle
        InitialGuess
        Tolerance
        MaxIterations
    end
    methods
        function obj = FixedPointMethod(func, x0, tol, maxIter)
            obj.FunctionHandle = func;
            obj.InitialGuess = x0;
            obj.Tolerance = tol;
            obj.MaxIterations = maxIter;
        end

        function [root, steps, errors] = solve(obj)
            k = 0;
            x_k = obj.InitialGuess;
            steps = [];
            errors = [];

            while k < obj.MaxIterations
                x_k1 = obj.FunctionHandle(x_k);
                steps = [steps; x_k1];
                errors = [errors; abs(x_k1 - x_k)];

                if abs(x_k1 - x_k) < obj.Tolerance
                    root = x_k1;
                    return;
                end

                x_k = x_k1;
                k = k + 1;
            end

            root = x_k1; % Return last computed value if max iterations reached
        end
    end
end
```

6.2 GUI code

```
function fixedPointGUI
    % Create the main figure
    f = figure('Position', [300, 300, 500, 400], 'MenuBar', 'none', ...
        'Name', 'Fixed Point Method', 'NumberTitle', 'off', ...
        'Resize', 'off');
```

```

% Create UI components
uicontrol('Style','text','Position',[20, 350, 100, 20], ...
    'String','Function f(x):');
funcInput = uicontrol('Style','edit','Position',[120, 350, 350, 20]);

uicontrol('Style','text','Position',[20, 310, 100, 20], ...
    'String','Initial Guess:');
x0Input = uicontrol('Style','edit','Position',[120, 310, 350, 20]);

uicontrol('Style','text','Position',[20, 270, 100, 20], ...
    'String','Tolerance:');
tolInput = uicontrol('Style','edit','Position',[120, 270, 350, 20]);

uicontrol('Style','text','Position',[20, 230, 100, 20], ...
    'String','Max Iterations:');
maxIterInput = uicontrol('Style','edit','Position',[120, 230, 350, 20]);

% Button to execute the method
uicontrol('Style','pushbutton','Position',[200, 180, 100, 30], ...
    'String','Solve','Callback',@solveCallback);

% Output text
outputText = uicontrol('Style','text','Position',[20, 140, 450, 20], ...
    'String','');

% Callback function
function solveCallback(~, ~)
    funcStr = get(funcInput, 'String');
    x0 = str2double(get(x0Input, 'String'));
    tol = str2double(get(tolInput, 'String'));
    maxIter = str2double(get(maxIterInput, 'String'));

    % Create function handle
    f = str2func(['@(x) ' funcStr]);

    % Create FixedPointMethod object
    fpm = FixedPointMethod(f, x0, tol, maxIter);

    % Solve and display result
    [root, steps, errors] = fpm.solve();
    set(outputText, 'String', ['Root: ' num2str(root)]);

    % Plot steps and errors
    figure('Name','Convergence Analysis','NumberTitle','off');

    subplot(2, 1, 1);
    plot(steps);
    title('Steps of Fixed Point Iteration');
    xlabel('Iteration');

    ylabel('Value');

```

```

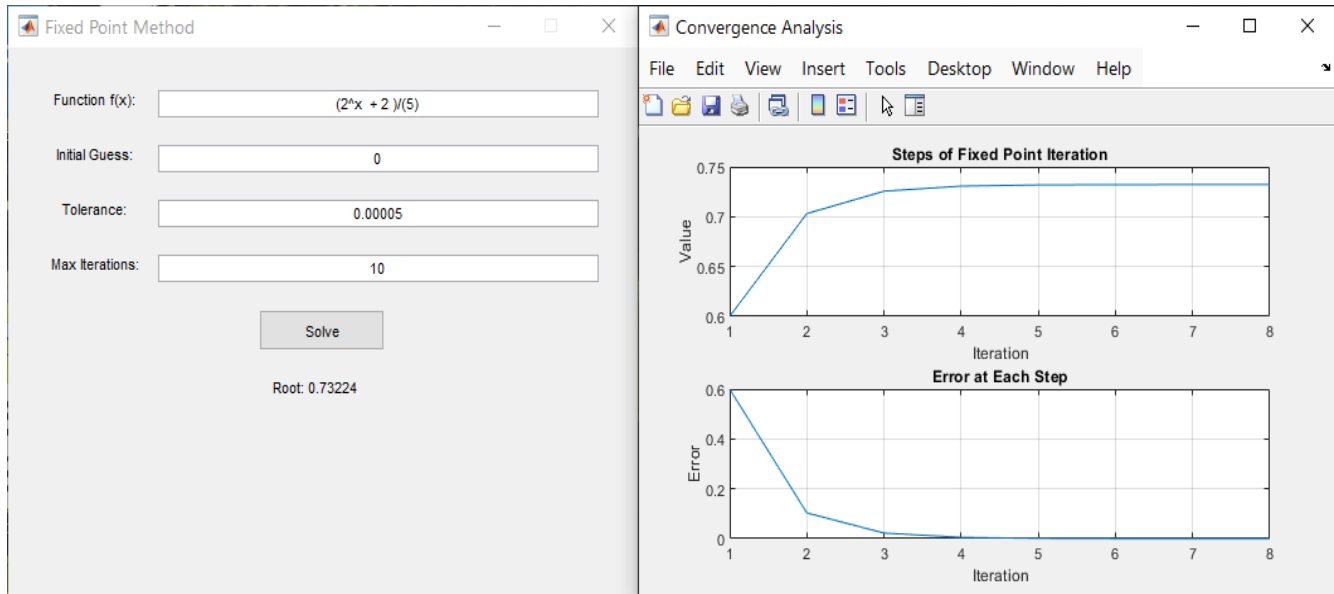
grid on;

subplot(2, 1, 2);
plot(errors);
title('Error at Each Step');
xlabel('Iteration');
ylabel('Error');
grid on;

end
end

```

6.3 figure and plot



CHAPTER 7: TRAPEZOIDAL METHOD

7.1 Class code

```
classdef TrapezoidalMethod
    properties
        FunctionHandle
        a
        b
        n
    end

    methods
        function obj = TrapezoidalMethod(func, a, b, n)
            obj.FunctionHandle = func;
            obj.a = a;
            obj.b = b;
            obj.n = n;
        end

        function integral = computeIntegral(obj)
            h = (obj.b - obj.a) / obj.n;
            S = 0;
            for i = 1:obj.n-1
                x_i = obj.a + i * h;
                S = S + obj.FunctionHandle(x_i);
            end
            integral = (h / 2) * (obj.FunctionHandle(obj.a) + 2 * S +
obj.FunctionHandle(obj.b));
        end

        function plotTrapezoids(obj)
            x = linspace(obj.a, obj.b, obj.n + 1);
            y = arrayfun(obj.FunctionHandle, x);
            plot(x, y, 'LineWidth', 2); % Plot the function
            hold on;
            for i = 1:obj.n
                fill([x(i), x(i), x(i+1), x(i+1)], [0, y(i), y(i+1), 0], 'r',
'FaceAlpha', 0.5);
            end
            hold off;
            title('Trapezoidal Method');
            xlabel('x');
            ylabel('f(x)');
            grid on;
        end
    end
end
```

7.2 GUI CODE

```
% GUI Implementation
function trapezoidalGUI()
    % Create the GUI
    uifig = figure('Position', [100, 100, 400, 300]);
    uicontrol('Style', 'text', 'Position', [50, 250, 300, 20], 'String',
'Trapezoidal Method GUI');
    uicontrol('Style', 'text', 'Position', [50, 210, 100, 20], 'String', 'Lower
Limit (a):');
    aInput = uicontrol('Style', 'edit', 'Position', [150, 210, 100, 20], 'String',
'0');
    uicontrol('Style', 'text', 'Position', [50, 180, 100, 20], 'String', 'Upper
Limit (b):');
    bInput = uicontrol('Style', 'edit', 'Position', [150, 180, 100, 20], 'String',
'1');
    uicontrol('Style', 'text', 'Position', [50, 150, 100, 20], 'String', 'Number
of Steps (n):');
    nInput = uicontrol('Style', 'edit', 'Position', [150, 150, 100, 20], 'String',
'10');

    uicontrol('Style', 'pushbutton', 'Position', [50, 100, 200, 30], 'String',
'Calculate Integral', ...
        'Callback', @(src, event) calculateIntegral(aInput, bInput,
nInput));
end

function calculateIntegral(aInput, bInput, nInput)
    a = str2double(get(aInput, 'String'));
    b = str2double(get(bInput, 'String'));
    n = str2double(get(nInput, 'String'));

    trapezoidal = TrapezoidalMethod(@(x) 2^x - 5*x + 2, a, b, n); % Example
function: f(x) = x^2
    integralValue = trapezoidal.computeIntegral();

    % Display result
    msgbox(['Calculated Integral: ', num2str(integralValue)], 'Result');

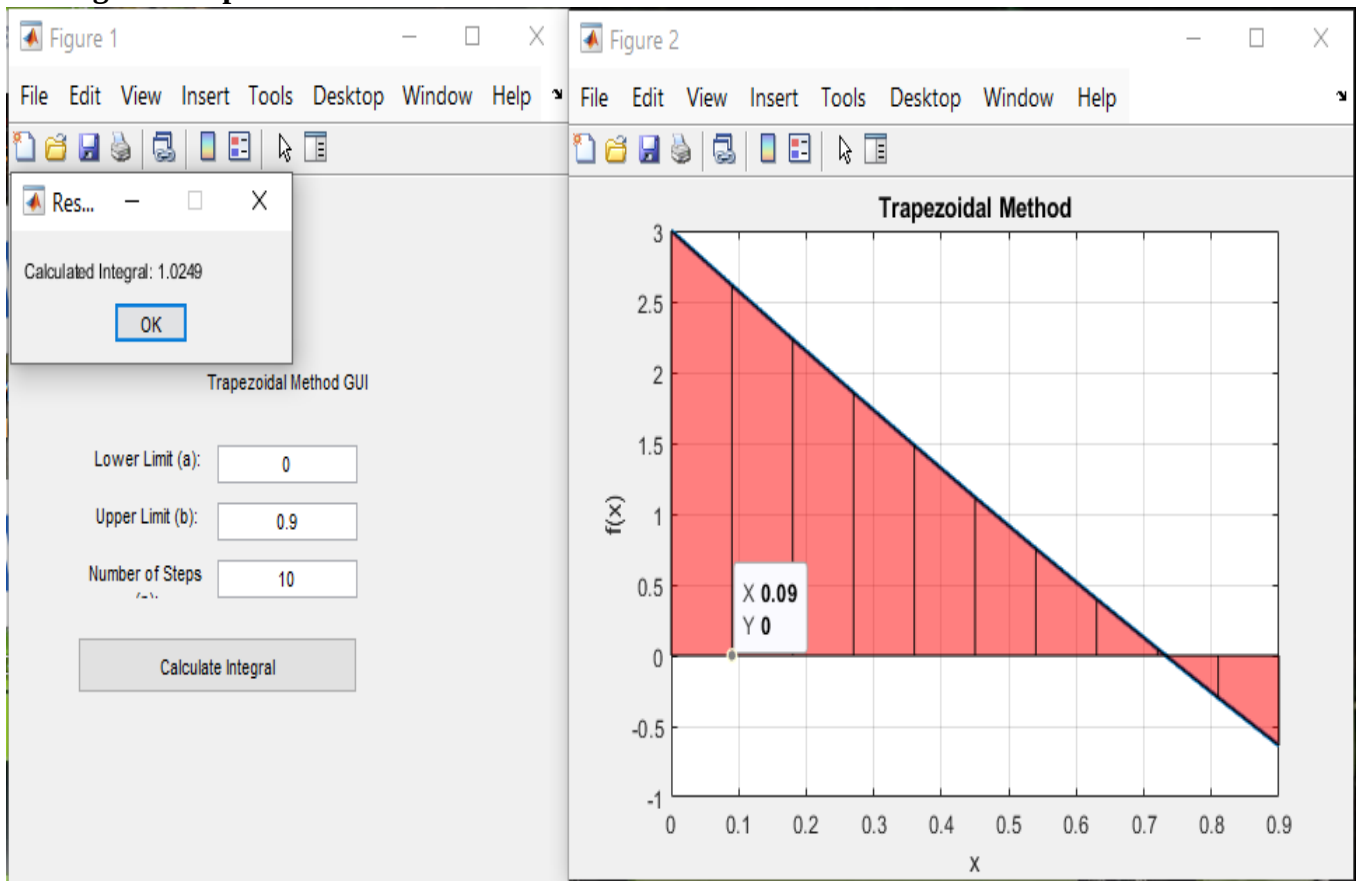
    % Plot trapezoids
    figure;
    trapezoidal.plotTrapezoids();
end
```

Lower limit = 0

Upper limit = 0.9

Answer appears in a message box

7.3 figure and plot



CHAPTER 8: CONCLUSION AND RECOMMENDATION

8.1 Recommendations for GUI Design

Prioritize Usability: Ensure that the interface is intuitive and easy to navigate. Conduct user testing to identify pain points and areas for improvement.

Consistency is Key: Maintain a consistent layout, color scheme, and typography throughout the application. This helps users build familiarity and reduces cognitive load.

Responsive Design: Design GUIs that adapt to different screen sizes and resolutions. This is crucial for applications accessed on various devices, including desktops, tablets, and smartphones.

Feedback Mechanisms: Provide clear feedback for user actions (e.g., button clicks, form submissions). This can include visual cues, sounds, or notifications to confirm actions.

Accessibility: Ensure that the GUI is accessible to all users, including those with disabilities. Follow accessibility guidelines (like WCAG) to make your application usable for everyone.

8.2 Conclusions on GUI Design

User-Centric Approach: The success of a GUI hinges on understanding user needs and behaviors. A user-centric design process leads to more effective and satisfying interfaces.

Importance of Testing: Regular usability testing is essential to identify issues and validate design choices. Engaging real users in the testing process provides invaluable insights.

Evolving Standards: GUI design is not static; it evolves with technology and user expectations. Staying updated with the latest trends and best practices is crucial for creating relevant and effective interfaces.

Integration of Technology: As technology advances (e.g., AI, voice interfaces), GUIs must adapt to incorporate new interaction methods, enhancing user experience and accessibility.

In summary, effective GUI design is a blend of usability, aesthetics, and continuous improvement, all centered around the user experience. By following best practices and remaining adaptable to change, designers can create interfaces that are not only functional but also enjoyable to use.

References

Books

1. "The Design of Everyday Things" by Don Norman

A classic book that discusses design principles applicable to GUIs and everyday objects.

2. "About Face: The Essentials of Interaction Design" by Alan Cooper, Robert Reimann, and David Cronin

This book provides insights into interaction design and user experience, focusing on GUI design.

3. "Don't Make Me Think" by Steve Krug

A practical guide to web usability that emphasizes intuitive design in GUIs.

4. "Designing Interfaces: Patterns for Effective Interaction Design" by Jenifer Tidwell

A comprehensive resource on interface design patterns and best practices for GUIs.